

PEVN — DOSSIER DE PRESENTACIÓN TÉCNICA

ARQUITECTURA DETALLADA, COMPONENTES, MODELOS DE DATOS Y ESTADO DE PREPARACIÓN (FASE 0.1)

Destinatarios: Comités Técnicos Evaluadores, Arquitectos de Software de MinTIC y MEN, Equipos de Ciberseguridad (ColCERT) e Ingenieros de Sistemas

Marco Metodológico: AI Software Factory v1.2 — Technical Evaluation Dossier

Fecha: 21 de Septiembre de 2026

Carácter: Read-Only Technical Assessment

1. Taxonomía de Evaluación Técnica Obligatoria

En este informe técnico, cada aseveración, subsistema o capacidad de PEVN se clasifica explícitamente en una o varias de las siguientes siete dimensiones taxonómicas:

- **[A] Implemented:** Código fuente estructurado, compilable y presente en el repositorio.
- **[B] Tested:** Verificado y validado mediante pruebas automatizadas (unitarias, integración o E2E).
- **[C] Architecturally Prepared:** Diseñado bajo patrones desacoplados que permiten la extensión sin rediseño estructural.
- **[D] Requires External Infrastructure:** Requiere servidores físicos, clústeres, proxies o hardware que debe ser provisto externamente.
- **[E] Requires Institutional Decision:** Depende de una directriz, acuerdo de gobierno o decisión del Ministerio o Secretaría de Educación.
- **[F] Requires Legal Review:** Requiere concepto o formalización por parte de asesores jurídicos del Estado.
- **[G] Not Yet Empirically Validated:** Funciona a nivel de diseño y pruebas locales, pero no ha sido probado con estrés de carga en producción.

2. Topología Arquitectónica General

PEVN adopta el patrón de **Micro-Monolito Modular Asíncrono** desacoplado de una **Single Page Application (SPA)** reactiva, complementado con bases de datos relacionales ACID y cachés en memoria:



• **Clasificación:** **[A] Implemented** | **[B] Tested** | **[C] Architecturally Prepared**.

3. Arquitectura del Backend

- **Tecnologías:** Python 3.12, FastAPI 0.110+, Uvicorn ASGI server, SQLAlchemy 2.0 (AsyncIO con **asyncpg**), Pydantic v2 Settings.
- **Estructura de Capas:**
- Capa de Controladores REST: **backend/app/api/v1/endpoints/** (28 controladores).

- Capa de Inyección de Dependencias y Seguridad: [backend/app/api/deps.py](#).
- Capa de Servicios de Dominio: [backend/app/services/](#) (18 servicios).
- Capa de Modelado ORM: [backend/app/models/](#) (24 archivos de modelo relacional).
- Capa de Auditoría Inmutable: [backend/app/audit/database_service.py](#).
- **Delimitación Transaccional:** Control explícito de confirmación a nivel de controlador (`await db.commit()`), garantizando atomicidad y previniendo rollbacks implícitos tras peticiones exitosas (verificado formalmente en B3-H01).
- **Clasificación:** [\[A\] Implemented](#) | [\[B\] Tested](#).

4. Arquitectura del Frontend

- **Tecnologías:** React 18.3.1, TypeScript 5.7.3, Vite 6.0.5, Tailwind CSS 3.4.17, React Router DOM 6.28.0, Axios 1.7.9.
- **Gestión de Estado y Seguridad de Tokens:** Estado en memoria volátil de React ([AuthContext](#)). **Cero almacenamiento en `localStorage`, `sessionStorage` ni `IndexedDB`**, como medida de mitigación frente al robo persistente de credenciales mediante XSS.
- **Soporte PWA:** Integración de Service Worker ([vite-plugin-pwa](#)) para almacenamiento en caché de activos estáticos y soporte de instalación en navegadores modernos.
- **Cliente HTTP:** Interceptores Axios que inyectan el token Bearer en memoria y manejan transparentemente errores 401 mediante una cola *single-flight refresh* llamando a [/api/v1/auth/refresh](#).
- **Clasificación:** [\[A\] Implemented](#) | [\[B\] Tested](#) | [\[C\] Architecturally Prepared](#).

5. Arquitectura de Base de Datos y Persistencia

- **Motor:** PostgreSQL 16.4 con driver asíncrono de alto rendimiento [asyncpg](#).
- **Evolución del Esquema:** 23 migraciones lineales gestionadas con Alembic ([backend/migrations/versions/](#)).
- **Modelo de Doble Usuario:**
- [pevn_admin](#): Propietario del esquema para ejecución de migraciones.
- [pevn_app](#): Usuario de mínimos privilegios para el runtime (`SELECT`, `INSERT`, `UPDATE`, `DELETE`), con privilegios `CREATE TABLE/SCHEMA` expresamente revocados.
- **Parámetros del Pool de Conexiones:** `pool_size = 10`, `max_overflow = 20`, `pool_recycle = 1800s`, `pool_pre_ping = True`.
- **Clasificación:** [\[A\] Implemented](#) | [\[B\] Tested](#).

Requerimiento para Producción: [\[D\] Requires External Infrastructure](#) (PgBouncer para conexión pooling masivo).

6. Autenticación y Criptografía de Credenciales

- **Algoritmo de Contraseñas:** Argon2id (RFC 9106), memoria de 64 MB (65.536 KiB), 3 iteraciones de tiempo, 4 hilos paralelos y salt criptográfico aleatorio de 16 bytes.
- **Tokens de Acceso (JWT):** Firmados con HMAC-SHA256 ([HS256](#)), caducidad estricta de 15 minutos (900 s), claims: [sub](#), [roles](#), [institution_id](#), [jti](#), [iat](#), [exp](#).
- **Tokens de Refresco (Refresh Tokens):** Almacenados como hash SHA-256 en la base de datos y entregados al cliente exclusivamente mediante cookies seguras:
- `HttpOnly = True`
- `SameSite = Strict`
- `Path = /api/v1/auth`
- `Secure = True` (en producción)
- Vigencia: 7 días.
- **Detección Activa de Replay / Reutilización:** Cada refresco emite un nuevo token y marca el anterior con [replaced_by_token_id](#). La reutilización de un token viejo revoca inmediatamente toda la familia ([family_id](#)), desconectando las sesiones activas y emitiendo alerta de auditoría crítica.
- **Protección contra Fuerza Bruta:** Bloqueo automático de cuenta por 15 minutos tras 5 intentos fallidos consecutivos.
- **Clasificación:** [\[A\] Implemented](#) | [\[B\] Tested](#).

7. Control de Acceso Basado en Roles (RBAC) y Jerarquía Anti-Escalamiento

- **Sintaxis Granular:** Formato **recurso:accion** (ej. **students:read**, **grades:create**, **evaluations:unlock_period**).
- **Comodines Soportados:** ***** (universal SuperAdmin), **users:*** (todas las acciones del recurso), ***:read** (lectura global).
- **Niveles de Seguridad:** Niveles numéricos del 10 al 100 que previenen escalamiento vertical de privilegios (un usuario de nivel N no puede crear ni gestionar usuarios con nivel $\geq N$).
- **Clasificación:** [A] Implemented | [B] Tested.

8. Aislamiento Multi-Inquilino (Multi-Tenancy) y Frontera Territorial DANE

- **Particionamiento Lógico:** Todas las tablas y consultas del dominio escolar filtran por **institution_id** extraído del token JWT verificado en el servidor.
- **Jerarquía DANE:** La función **scope_contains()** valida matemáticamente que un usuario pertenezca a la misma institución o tenga jurisdicción territorial superior (Nacional o Departamental).
- **Protección Anti-IDOR (Blind 404):** Intentos de manipulación de identificadores UUID en URLs devuelven **404 Not Found** en lugar de **403 Forbidden**, evitando que atacantes descubran qué identificadores existen en otras instituciones.
- **Clasificación:** [A] Implemented | [B] Tested.

9. Auditoría Inmutable y Censura de Metadatos

- **Tabla Inmutable:** **audit_logs** en PostgreSQL con índices optimizados por **event_type**, **actor_id**, **institution_id**, **correlation_id** y **occurred_at**.
- **Sanitización de Datos Sensibles (Zero-Secrets):** Función recursiva **sanitize_audit_metadata()** que censura automáticamente con **[REDACTED]** cualquier clave que contenga contraseñas, tokens JWT, secretos, sales o cookies.
- **Clasificación:** [A] Implemented | [B] Tested.

10. Arquitectura de API RESTful

- **Estándar:** RESTful JSON bajo especificación OpenAPI v3.
- **Versionamiento:** Prefijo **/api/v1/** en todos los controladores.
- **Manejo de Errores Global:** Respuestas de error canónicas sanitizadas con código de error, mensaje comprensible y UUID de correlación (**X-Correlation-ID**), sin exponer trazas de pila en producción.
- **Clasificación:** [A] Implemented | [B] Tested.

11. Arquitectura de Aulas Virtuales (BigBlueButton)

- **Abstracción:** Protocolo **IMeetingProvider** con implementaciones **BBBAdapter** (cliente HTTP asíncrono con firma SHA-1/256) y **MockMeetingProvider**.
- **Fábrica de Proveedores:** **MeetingProviderFactory** conmuta según **MEETING_PROVIDER_TYPE**.
- **Telemetría y Control de Asistencia:** Endpoints **/launch**, **/join**, **/leave** y **/end** con cálculo automático de duración en segundos.
- **Control de Grabaciones:** Sincronización asíncrona y visibilidad restringida por el docente.
- **Clasificación de Software:** [A] Implemented | [B] Tested.
- **Clasificación de Infraestructura:** [D] Requires External Infrastructure — El servidor físico BigBlueButton y el servidor de relay Coturn (STUN/TURN en TCP 443) NO han sido comisionados.

12. Arquitectura de Almacenamiento de Archivos (Storage)

- **Implementación Actual:** **LocalStorageService** en disco local (**backend/data/storage/{institution_id}/...**).
- **Seguridad en Carga de Archivos:** Detección de Magic Bytes, rechazo de ejecutables, límite de 10 MB y mitigación de Path Traversal.
- **Clasificación:** [A] Implemented | [B] Tested.

- **Requerimiento para Producción:** **[C] Architecturally Prepared** | **[D] Requires External Infrastructure** — Requiere implementación del adaptador **S3StorageService** (AWS S3 / MinIO) para almacenamiento de objetos en la nube.

13. Arquitectura de Escalabilidad y Concurrencia

- **Backend Stateless:** Escalamiento horizontal mediante réplicas detrás de un balanceador de carga.
- **Frontend SPA:** Distribuible en CDN estática.
- **Clasificación:** **[C] Architecturally Prepared**.
- **Estado de Validación:** **[G] Not Yet Empirically Validated** — La arquitectura está diseñada para permitir escalamiento horizontal; la capacidad efectiva a escala nacional deberá determinarse mediante pruebas de carga y validación de infraestructura.

14. Interoperabilidad con Sistemas del Estado (SIMAT / DUE / DANE)

- **Alcance y Compatibilidad:** PEVN incorpora estructuras de datos y reglas de negocio compatibles con el modelo de matrícula correspondiente y se encuentra arquitectónicamente preparada para una eventual interoperabilidad, sujeta a las interfaces técnicas oficiales, autorizaciones y acuerdos institucionales que determine la entidad competente.
- **Límites Factuales:** PEVN no reemplaza al SIMAT, al DUE ni a los sistemas del DANE, sino que armoniza con sus catálogos y estructuras de datos. La sincronización en vivo mediante servicios web depende de acuerdos formales y provisión de credenciales por parte del MEN.
- **Clasificación:** **[A] Implemented** (Modelos y reglas locales) | **[C] Architecturally Prepared** | **[E] Requires Institutional Decision** | **[F] Requires Legal Review**.

15. Matriz de Síntesis de Componentes Técnicos

Componente Técnico	[A] Impl	[B] Test	[C] Arch	[D] Infra	[E] Inst	[F] Legal	[G] Not Emp
---	---	---	---	---	---	---	---
Autenticación (Argon2id + JWT)	■	■	■	—	—	—	—
Control de Acceso (RBAC + Scope)	■	■	■	—	—	—	—
Aislamiento Multi-Tenant (Anti-IDOR)	■	■	■	—	—	—	—
Catálogo Oficial DANE / DUE	■	■	■	—	—	—	—
Padrón SIMAT y Matrículas	■	■	■	—	—	—	—
Portal Docente y Actividades	■	■	■	—	—	—	—
Submissions (Entregas Alumnos)	■	■	■	—	—	—	—
Evaluación SIEE (Decreto 1290)	■	■	■	—	—	—	—
Convivencia (Ley 1620 / Obs.)	■	■	■	—	—	—	—
Circulares y Acuse Digital	■	■	■	—	—	—	—
Software Clases Virtuales	■	■	■	—	—	—	—
Servidor Físico BigBlueButton	—	—	■	Sí	—	—	—
Servidor Coturn (STUN/TURN)	—	—	■	Sí	—	—	—
Almacenamiento S3 / MinIO	—	—	■	Sí	—	—	—
Generador Masivo de PDFs	—	—	■	—	—	—	—
Notificaciones Push / SMTP	—	—	■	Sí	—	—	—
Enlace en Vivo API SIMAT	—	—	■	—	Sí	Sí	—
Escalabilidad Masiva Nacional	—	—	■	Sí	—	—	Sí
Política de Datos de Menores	—	—	—	—	Sí	Sí	—
Convenio de Donación al Estado	—	—	—	—	Sí	Sí	—